

CS & IT ENGINEERING



Data Structure & Programming

Stack and Queues

Lecture No.- 03



By- Satya sir

Recap of Previous Lecture



- Stack Permutations

$$\begin{cases} \text{Static sequence of Pushing} & : \frac{2^n C_n}{n+1} \\ \text{Dynamic sequence of Pushing} & : n! \end{cases}$$

- Applications of Stack

- Expression

- Infix Notation

- Prefix Notation : Polish Notation

- Post-fix Notation : Reverse Polish Notation

Topics to be Covered



- Expression Conversion

- Infix to Postfix

- Infix to Prefix

- Expression Evaluation

- Postfix Exp

- Prefix Exp





Topic : Expression Conversion



Infix to Postfix Conversion

Let 'X' be given Infix Expression and 'Y' be resultant Postfix Exp. 'S' be an Empty Stack initially.

① Scan 'X' from Left to right one token at a time.

② If Token == Operand, add it to 'Y'.

③ If Token == '(', Push it on to 'S'.

④ If Token == Operator [OP_s , scanned operator], check Top of Stack (TOS)

a) If Stack is Empty : Push OP_s on to 'S'.

b) If TOS == '(' : Push OP_s on to 'S'

c) If TOS == Operator [OP_t , Top Operator]

4.c. i) If $OP_t \geq OP_s$, Pop it, add to Y,
Repeat until $OP_t < OP_s$, Push OP_s on to S.

c. ii) If $OP_t < OP_s$, : Push OP_s on to S.

⑤ If Token == ')'

a) Pop from S,

b) Add to Y until first '(' is encountered

c) ignore ()

⑥ Repeat Steps ② to ⑤ until complete 'X' is Scanned.

⑦ Once 'X' is Scanned Make Stack Empty

⑧ 'Y' is Resultant Postfix Exp. [Pop, add to Y].



Topic : Expression Conversion

$\wedge, (*, /), (+, -)$
 High $\rightarrow$ Low



Ex: Convert Infix Expression $X: (a + b * c / (d - e) \wedge f / (g * h))$ to Postfix Expression.

Scanned Token: '(', a, +, b, ^{ops}*, c, ^{ops}/, (, d, -, e,), ^{ops}^, f, /, (, g, *, h,)

Stack, S :

	^{ops}	^{ops}										
/	+	*	/	/	-	/	^	/	/	*	/	/
	Pop	Pop	Pop		Pop		Pop	Pop		Pop		

Postfix exp, Y: a b c * d e - f ^ g h * / +



Topic : Expression Conversion



Ex:2

Infix Exp :

$7 + 3 / 2 * 4 - 5 * 2 / 6 + 8 \wedge 1 * 3$

Convert to Postfix:

Stack, S

+	/	*	-	*	/	+	$\wedge$	*
Pop	Pop	Pop	Pop	Pop	Pop	Pop	Pop	Pop

Postfix, Y : $7 3 2 / 4 * + 5 2 * 6 / - 8 1 \wedge 3 * +$



Topic : Expression Conversion



Ex: Convert Infix Expression, $X = a * (b \wedge c / d \wedge (e * f + g / h) - e)$ to Postfix Expression

Stack, S

*	/	$\wedge$	/	$\wedge$	/	*	+	/	+	-	/
Pop		Pop	Pop	Pop		Pop	Pop	Pop		Pop	

y: $a b c \wedge d e f * g h / + \wedge / e - *$



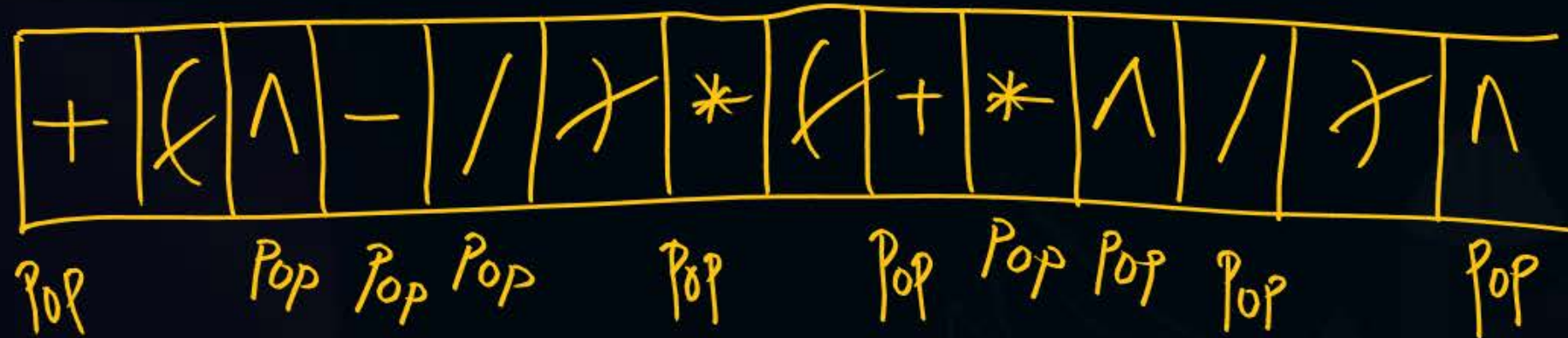
Topic : Expression Conversion



Infix Expression: $P + (Q \wedge R - S / T) * (U + V * W \wedge X / Y) \wedge Z$

Postfix Expression: $PQR \wedge ST / - UVWX \wedge * Y / + Z \wedge * +$

Stack





Infix to Prefix Conversion

Let, 'X' be given Infix Exp, 'Y' be Intermediate Exp and 'Z' be resultant Prefix Exp, 'S' be an Empty Stack.

① Scan 'X' from Right to Left, ')' == '(' and '(' == ')'

② Follow the Procedure of Infix to Postfix conversion, except

If $OP_t > OP_s$ Pop OP_t , add to Y
Repeat Until $OP_t \leq OP_s$
Push OP_s on to S.

If $OP_t \leq OP_s$ Push OP_s on to S.

③ Reverse obtained Intermediate Expression, $Y == Z$, Resultant Prefix Expression.

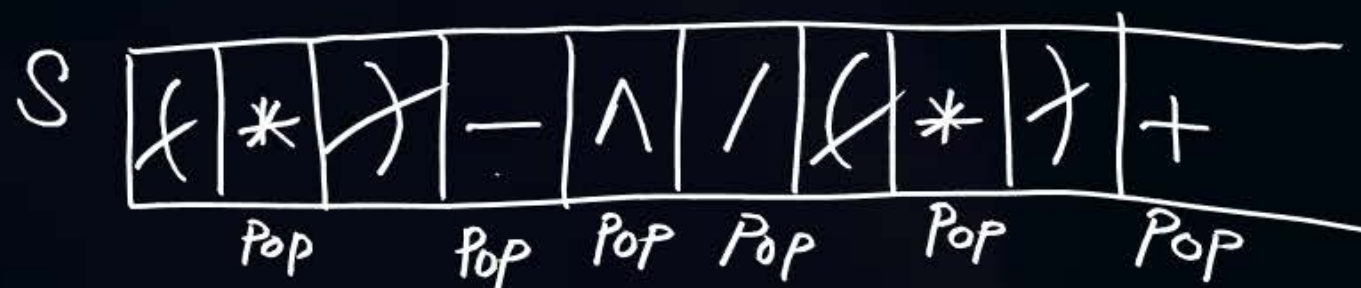


Topic : Expression Conversion

Ex:-1

Infix Exp: $a + (b * c) / d \wedge e - (f * g)$

Prefix Exp: $- + a / * b c \wedge d e * f g$



Y: $gf * ed \wedge cb * / a + -$



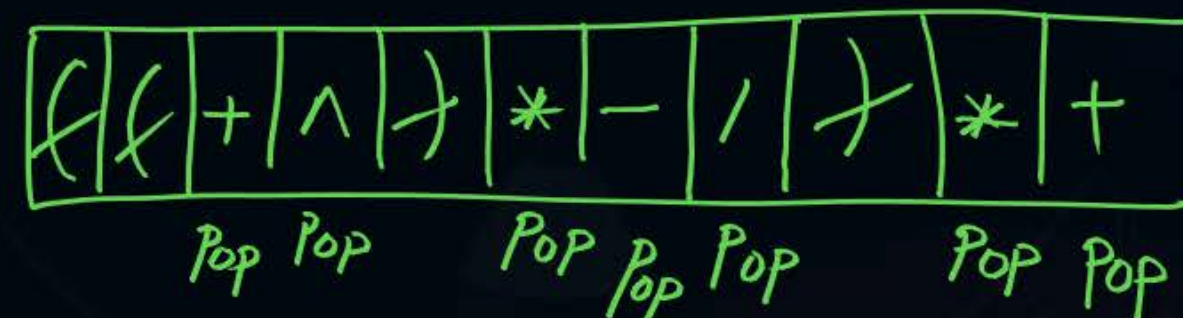
Ex:2

Infix Exp: $7 + 3 * (2 / 5 - 4 * (6 \wedge 2 + 3))$

Prefix Exp:
(Z)

$+ 7 * 3 - / 2 5 * 4 + \wedge 6 2 3$

Stack(s)



Y: $3 2 6 \wedge + 4 * 5 2 / - 3 * 7 +$



Expression Evaluation

Postfix Exp Evaluation

- ① Scan given Postfix Exp from Left to Right
- ② Token == Operand, Push it on to S.
- ③ If Token == Operator, Then
 - (i) Pop first operand $[OP_1]$
 - (ii) Pop second operand $[OP_2]$
 - (iii) Perform $[OP_2 \text{ operator } OP_1]$ and Push back.
- ④ Repeat steps ② & ③ until Exp Completes.
- ⑤ Pop the Result.

Prefix Exp Evaluation

- ① Scan from Right to Left
- ② If Token == Operand, Push it on to S.
- ③ If Token == Operator, Then
 - (i) Pop first operand (OP_1)
 - (ii) Pop second operand (OP_2)
 - (iii) Perform $[OP_1 \text{ operator } OP_2]$ and Push back.
- ④ Repeat steps ② and ③ until Exp Completes.
- ⑤ Pop the Result.



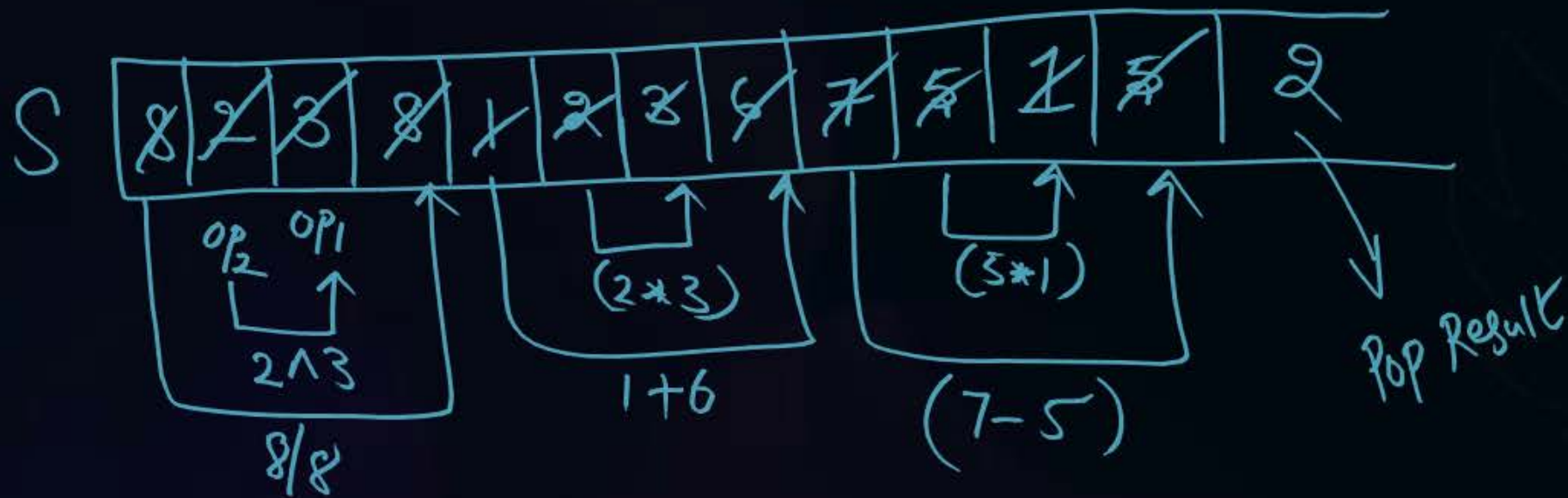
Topic : Expression Evaluation



Example : (ISRO 2016)

The Result after Evaluation of

Post fix Exp: $8\ 2\ 3\ \wedge / 2\ 3\ * + 5\ 1\ * -$ is 2





Topic : Expression Evaluation



Ex: [GATE 2015]

The Result Evaluating Postfix Exp

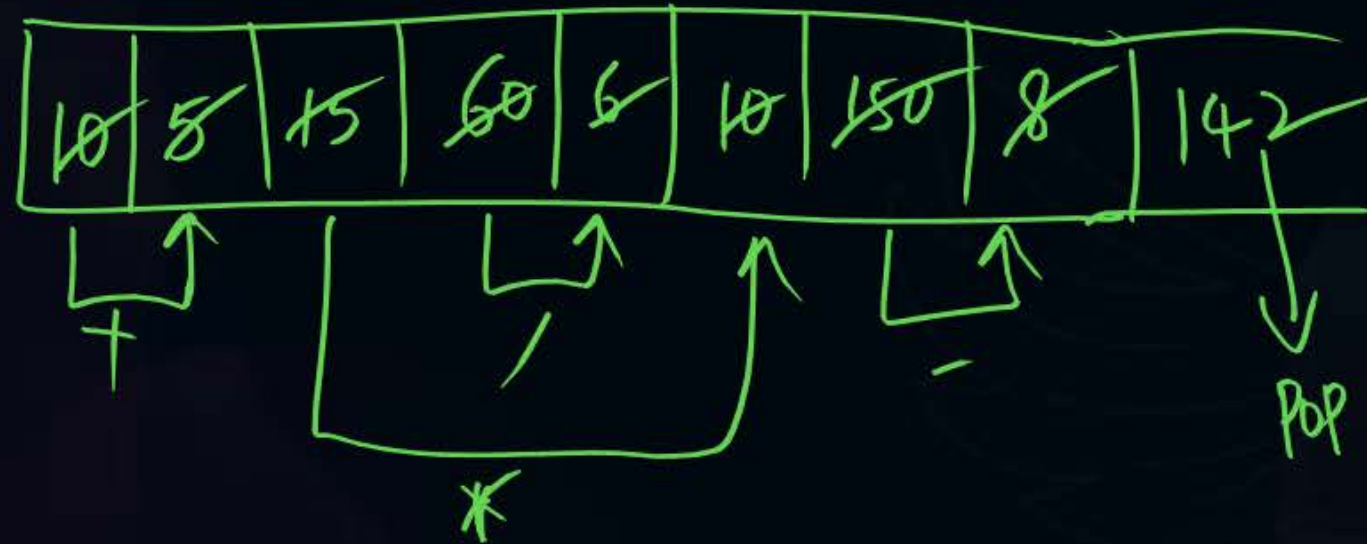
10 5 + 60 6 / * 8 — is —

A) 284

B) 213

☒ C) 142

D) 71





Topic : Expression Evaluation



H/w

Convert Infix Exp: $7 + (3 * 5) - (9 / 2) + (4 \wedge 2) - 8$

to Postfix Exp=

_____ ,

Evaluate as Postfix

Prefix Exp=

_____ ,

Evaluate as Prefix



2 mins Summary



- Expression Conversion
- Expression Evaluation.

To be Contd...





THANK - YOU